PROJECT DOCUMENTATION

Hotel Management System

A single-property hotel platform — guest-facing site, staff operations dashboard, and admin CMS — built on Next.js 15 and Supabase. This guide walks through every feature and how to use it.

STACK
Next.js 15 · Supabase · Tailwind · shadcn/ui

STATUS
Phases 1–7 complete

PAYMENTS
Khalti · eSewa · Pay-at-hotel

DEPLOYMENT
Vercel + Supabase Pro

VERSION
v1 (pre-launch)

DOCUMENT DATE
2026-05-27

Contents

1. Overview

The Hotel Management System is a complete, end-to-end platform for a single boutique property. It bundles three experiences into one codebase:

Public website

A boutique hero, image-led room listings, a menu, services, FAQs, and Google-cached reviews — all driven by a database-backed CMS.

Booking flow

Browse → pick dates → live total preview → email-OTP verification → confirmation. No guest signup required.

Staff dashboard

Today's arrivals/departures, check-in / check-out, walk-in bookings, service requests, live chat with guests, reports.

Admin CMS

Site settings, branding, pages, gallery, FAQs, amenities, testimonials, email templates, staff invites, audit log.

The system is intentionally **opinionated and small**: one property, English-only text, manual refunds, no in-app reviews, no dynamic pricing. The "What NOT to do" guard rails are documented in §9.

2. Architecture & tech stack

LAYER	CHOICE	NOTES
Framework	Next.js 15 (App Router)	React 19, TypeScript, server actions
Styling	Tailwind CSS + shadcn/ui	Boutique terracotta + sand + copper palette
Type fonts	Inter + Playfair Display	Loaded via `next/font`
Database	Supabase Postgres	Row-Level Security enforced everywhere
Auth	Supabase Auth — email OTP only	No password, no SMS, no guest signup
Storage	Supabase Storage	Bucket: `public-images`
Realtime	Supabase Realtime	Used for guest ↔ staff chat
Email	Resend	Transactional + OTP
Validation	Zod	Server-action + webhook entry points
Payments	Khalti + eSewa + pay-at-hotel	Khalti/eSewa scaffolded, online flow deferred
Hosting	Vercel + Supabase Pro	Vercel Cron refreshes Google reviews daily

Key directories

```
app/                    Next.js App Router (public, auth, staff, admin)
  (auth)/               Login + verify-otp
  (staff)/dashboard/    Reception + manager operations
  (admin)/admin/        Super-admin CMS
  rooms/, menu/, ...    Public pages
components/             UI primitives + shell + public + chat
lib/
  supabase/             Browser, server, admin, middleware clients
  pricing.ts            Total calculation
  availability.ts       Date-range room search
  cancellation.ts       Refund tier matching
  email.ts              Resend wrapper
  audit.ts              writeAudit() helper
  validation/           Zod schemas grouped by domain
supabase/
  migrations/           0001–0007 SQL migrations
middleware.ts           Role-based route gate
types/database.ts       Generated Supabase types (stub by default)
```

Database highlights

- **Decoupled profiles.** `profiles` stays separate from `auth.users`; a trigger links new auth users back to existing stub profiles by email — supports walk-in guests without email accounts.

- **Atomic availability.** A `btree_gist` exclusion constraint on `bookings (room_id, daterange(check_in, check_out))` rejects double-bookings at the database level.
- **Snapshotted totals.** `bookings.total_amount` is computed at booking time so a later `base_price` change does not silently re-price old bookings.
- **Audit log on every mutation.** Server actions call `writeAudit`; super admins see the full reverse-chronological log with old → new diffs.

3. Roles & access

ROLE	ROUTES ACCESSIBLE	CAPABILITIES
guest	Public pages, `/booking/[id]`, `/my-bookings`, `/chat`	Browse, book, cancel own booking, request services, chat with reception
receptionist	`/dashboard/**`	Check guests in/out, walk-in bookings, handle service requests, reply in chat
manager	All staff routes + `/dashboard/rooms`, `/dashboard/menu`, `/dashboard/reports`, `/dashboard/cancellations`	Receptionist capabilities + edit rooms, menu, services, run reports, record refunds
super_admin	Everything + `/admin/**`	Full CMS control — settings, branding, pages, gallery, templates, staff invites, audit log

> Defence in depth. Routes are gated three ways: the middleware blocks at the edge, layout files re-check the role, and every mutating server action verifies the caller. Disabled accounts (`is_active=false`) are kicked out even if their session is still valid.

4. Getting started — local setup

You need Node 20+, a Supabase project, and (optionally) a Resend API key for emails.

1. **Install dependencies.**

```
npm install
```

2. **Create a Supabase project** at `https://supabase.com` and copy three values:
   - 3 Project URL → `NEXT_PUBLIC_SUPABASE_URL`
   - 4 anon public key → `NEXT_PUBLIC_SUPABASE_ANON_KEY`
   - 5 service role key → `SUPABASE_SERVICE_ROLE_KEY`
6. **Copy env file** and fill in values.

```
cp .env.example .env.local
```

Required at minimum: the three Supabase keys, `SESSION_COOKIE_SECRET` (any random string), and `NEXT_PUBLIC_SITE_URL`.

7. **Apply migrations.** Run each SQL file in `supabase/migrations/` in order — through the Supabase Dashboard SQL editor, or via the CLI:

```
supabase db push
```

8. **Promote yourself to super admin.** Create a user in the Supabase dashboard, then in the SQL editor run `update profiles set role = 'super_admin' where email = 'you@example.com';`

9. **Verify env.** Sanity-check everything is wired up:

```
npm run check
```

10. **Boot the dev server.**

```
npm run dev
```

The app runs on `http://localhost:4000`.

> Auth template note. Supabase ships a magic-link email template by default. Switch it to a 6-digit OTP under Auth → Email Templates → Magic Link (use `{{ .Token }}`). Also wire Resend as the SMTP provider so emails go out with your domain.

5. For guests — using the public site

> Guests never sign up. They browse anonymously and verify their email only at the moment of booking confirmation. A returning guest can sign in to see `/my-bookings`.

5.1 Browsing

- `/` — boutique homepage rendered from the `pages` + `page_sections` tables. Sections can be hero, text, gallery, CTA, or FAQ.
- `/rooms` — image-led grid of every active room type with price, max guests, and amenities.
- `/rooms/[slug]` — detail page with a 5-image asymmetric gallery and a sticky booking card on desktop.
- `/menu` — browse food and beverages grouped by category. **Not** a cart — read-only.
- `/services` — list of services offered (housekeeping, laundry, transport, etc).
- `/reviews` — Google Reviews cached in the database, with an average-rating chip.

5.2 Booking flow

1. Pick a room type at `/rooms/[slug]`; fill check-in, check-out, guest count, name, email, phone.
2. Choose payment method — **Pay at hotel** (no online step) or **Pay online** (deferred to a future Khalti/eSewa flow).
3. Submit → server re-validates availability and stashes a signed intent cookie.
4. An OTP email arrives via Resend. Enter the 6-digit code at `/verify-otp`.
5. The booking row is inserted and you land on `/booking/[id]` with a printable receipt.

> **Double-booking is impossible.** Even if two guests submit at the same millisecond, the Postgres exclusion constraint on `(room_id, daterange)` rejects the second one. The second guest sees "no rooms available".

5.3 After booking

- `/my-bookings` — list of all bookings tied to the signed-in email.
- `/booking/[id]` — receipt with status, payment status, dates, total, and a **Cancel** button while status is pending or confirmed.
- **Request services** — once a booking is confirmed or checked-in, a service-request widget appears on the booking page.
- **Chat** at `/chat` — a live conversation with reception over Supabase Realtime. The unread counter resets when you open the page.

5.4 Cancellation & refunds

Refunds follow the policy stored in `cancellation_policy`:

HOURS BEFORE CHECK-IN	REFUND TIER
> 72h	100% of paid amount
24–72h	50% of paid amount
< 24h	0% (forfeit)

The system computes `refund_amount_due` automatically. Actual refunds are processed manually by a manager outside the app (Khalti/eSewa dashboards or cash) and recorded on `/dashboard/cancellations` — see §7.3.

6. For staff — reception dashboard

> Sign in at `/login` with your staff email. After OTP verification you land on `/dashboard` — six metric tiles plus a preview of today's arrivals.

6.1 Dashboard home

- Six **metric tiles**: today's arrivals, today's departures, occupancy %, open chats, pending refunds, open service requests.
- Below the tiles: a list of today's arrivals with check-in actions one click away.

6.2 Bookings (`/dashboard/bookings`)

The operational heart of the dashboard. Four sections:

1. **Today's arrivals** — each row has a **Check in** button that atomically flips the booking to `checked_in` and the room to `occupied`.
2. **Today's departures** — each row has a **Check out** button that flips the booking to `checked_out` and the room to `cleaning`.
3. **Cleaning queue** — rooms in `cleaning` status. The **Mark ready** button returns them to `available`.
4. **Recent bookings** — last 20 entries with status + payment badges, useful for quick lookup.

6.3 Walk-in (`/dashboard/walk-in`)

For guests who arrive without a reservation. Receptionists can:

- Pick dates and a room type — the system finds an available room.
- Enter guest details (email *optional* — a stub profile is created if missing).
- Optionally check the guest in immediately (room flips to `occupied` in the same action).
- Mark as paid at desk — creates a `payments` row with the chosen method.

Walk-in bookings carry `verification_method = 'staff_call'` and the staff member's ID in `verified_by`.

6.4 Service requests (`/dashboard/service-requests`)

A simple Kanban-style status flow: **received** → **in progress** → **completed** (or **cancelled**). Each transition is audit-logged.

6.5 Chat (`/dashboard/chat`)

- Inbox of all guest conversations sorted by `last_message_at` descending.
- Unread conversations show an accent border and a count badge.
- Click into `/dashboard/chat/[conversationId]` for the live message thread (Supabase Realtime — no refresh needed).
- Viewing the conversation resets the staff unread counter to 0 via a trigger.

7. For managers — operations & reports

> Managers see everything receptionists see, plus rooms / menu / services CRUD, the reports page, and the cancellations register.

7.1 Rooms (`/dashboard/rooms`)

A single page with two tables — **room types** (Standard / Deluxe / Suite by default) and **rooms** (individual numbered units linked to a type).

- Create / edit / delete room types: name, slug (auto-generated), description, base price, max guests, amenities, images.
- Create / edit / delete rooms: room number, type, floor, status (`available` / `occupied` / `cleaning` / `out_of_service`).
- **Price changes only affect new bookings.** Existing bookings keep their snapshotted `total_amount` .

7.2 Menu & services

- `/dashboard/menu` — food items: name, description, price, category, availability toggle.
- `/dashboard/services-manage` — services: name, description, price, category, availability toggle.
- Both feed straight into the public-facing `/menu` and `/services` pages.

7.3 Cancellations & refunds (`/dashboard/cancellations`)

1. The page lists every cancelled booking in two buckets — **refund pending** and **refund recorded**.
2. For pending rows: open the row, see the computed `refund_amount_due` , process the refund out-of-band, then click **Record refund** with the actual amount + reference.
3. The action updates `refunded_amount` , `refund_reference` , `refunded_at` , flips `payment_status` to `refunded` (or `partially_refunded`), fires the `booking_refunded` email, and writes an audit row.

7.4 Reports (`/dashboard/reports`)

A snapshot — not analytics. The page shows:

Bookings
Total · confirmed · cancelled (rate %).

Revenue
All-time gross from `payments` .

Occupancy
Snapshot % with a horizontal progress bar.

Top room types
Booked counts with relative-width bars.

Charts and date filters are intentionally deferred.

8. For super admins — CMS & system

> The `/admin` section is super-admin-only. All edits are written to `audit_logs` ; mistakes can be inspected (and corrected) any time.

8.1 Settings (`/admin/settings`)

Hotel name, tagline, address, contact, currency (default NPR), timezone (default Asia/Kathmandu), tax rate (default 13%), service rate (default 10%), Google Place ID. The homepage reads `hotel_name` live — no redeploy.

8.2 Branding (`/admin/branding`)

Primary / secondary / accent colors, font family. A live preview swatch lets you see changes before saving. *Note:* in v1 the boutique palette ships locked; branding fields are documentation-only.

8.3 Pages & sections

- `/admin/pages` lists the four fixed pages: **home**, **about**, **contact**, **terms**.
- Open a page to edit metadata and manage its sections. Five section types: `hero`, `text`, `gallery`, `cta`, `faq`.
- Each section has a Zod-validated form. Drag-via-sort-order, toggle visible, delete.

8.4 Gallery (`/admin/gallery`)

Image upload backed by Supabase Storage. 10MB cap, allowlist of image MIME types. Storage object is rolled back if the DB insert fails; on delete, the storage object is best-effort removed too.

8.5 FAQs, amenities, testimonials

Three near-identical CRUD pages. Each row has visible/hidden toggles, per-row edit/delete, and an audit log entry on every mutation.

8.6 Email + notification templates (`/admin/templates`)

Inline editor for subject, body, and active state for every template. Templates use `{{var}}` placeholders rendered at send time by `lib/email-from-template.ts`. Default templates include:

- `otp_login` — sign-in code
- `booking_confirmation` — sent on booking insert
- `booking_cancelled` — sent when status flips to cancelled
- `booking_refunded` — sent when a refund is recorded
- `review_request` — manual send (not auto-triggered in v1)

8.7 Google Reviews (`/admin/reviews`)

Shows the configured Place ID, last refresh timestamp, and a row count. A **Refresh now** button hits the Places API immediately. A Vercel Cron at `02:00 UTC` daily keeps the cache fresh automatically — gated by `CRON_SECRET`.

8.8 Staff (`/admin/staff`)

1. List of every non-guest profile with role, active status, last sign-in.

2. **Invite staff** form — creates a stub profile with the target role, then sends a Supabase Auth invite. When the invitee accepts, the trigger links their auth user to the stub.
3. **Change role** and **toggle active** actions with safeguards: you cannot demote yourself or disable yourself.

8.9 Audit log (`/admin/audit`)

Reverse-chronological table of every mutation. Filter by action, entity type, actor email, or date range. Click a row to expand the JSON diff (old → new) inside a `<details>` dropdown.

9. Out of scope — v1 constraints

> These are intentional simplifications. Re-read this list before adding features.

NOT IN V1	WHY
No SMS OTP	Email-only via Resend. Phone is collected for staff contact only.
No guest signup	OTP gate appears only at booking confirmation.
No in-app food ordering	The menu page is browse-only CMS content, not a cart.
No in-app review writing	Reviews live on Google; we cache them via the Places API.
No dynamic pricing	One `base_price` per room type. No weekend uplifts, no calendar overrides.
No automatic refund	System computes the recommended amount; admin processes it out-of-band.
No multi-language	English-only text columns in v1.
No page builder	Sections limited to 5 predefined types. No custom HTML/JS injection.
No multi-property	Single hotel only — no `properties` table.
No Stripe	Khalti + eSewa only at v1. Stripe deferred for foreign cards.

10. Useful commands & references

NPM scripts

COMMAND	WHAT IT DOES
`npm run dev`	Dev server on `http://localhost:4000`
`npm run build`	Production build
`npm run start`	Run the production build on `:4000`
`npm run lint`	ESLint (Next 15 flat config)
`npm run type-check`	`tsc --noEmit`
`npm run db:types`	Regenerate Supabase types (requires linked CLI)
`npm run check`	Verify env vars + Supabase connectivity

Environment variables

VAR	REQUIRED	PURPOSE
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Anon public key
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Server-side admin key
`SESSION_COOKIE_SECRET`	Yes	HMAC secret for signed booking intent cookies
`NEXT_PUBLIC_SITE_URL`	Yes	Base URL for absolute links
`RESEND_API_KEY`	For emails	OTP + transactional emails. If unset, emails are no-op'd with a log line.
`RESEND_FROM_EMAIL`	For emails	From-address for outgoing mail
`GOOGLE_PLACES_API_KEY`	For reviews	Powers the reviews cache
`CRON_SECRET`	For cron	Gates `/api/cron/refresh-google-reviews`
`KHALTI_*` / `ESEWA_*`	Deferred	Online payment keys (flow not wired in v1)

Reference files inside the repo

- `README.md` — quick-start setup.
- `BUILD_PLAN.md` — phase-by-phase progress log, source of truth.

- `hotel-system-architecture.docx.pdf` — original architecture spec. When this guide and the doc disagree, the doc wins.
- `supabase/README.md` — how to apply migrations + how to bootstrap a super admin.

Generated 2026-05-27 from project sources. For implementation detail beyond this guide, consult `BUILD_PLAN.md` and the architecture PDF.